Vector Database Engine

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a small vector database that stores fixed-dimension vectors of floating-point numbers and answers *nearest-neighbor* queries against them. Given a query vector, the database returns the k stored vectors that are most similar to it. "Most similar" is measured by Euclidean distance: the smaller the distance, the more similar the vectors. The result is a network server that listens on a TCP port and a small command-line client that exercises it.

Vector databases are a different kind of database from the relational store you built in Project 01. They do not store rows with named columns; they store points in a high-dimensional space. They do not answer queries like "which users are from Pakistan;" they answer queries like "which 10 stored points are closest to this new point." This is the operation that powers image search, music recommendation, semantic text search, and the retrieval step in modern AI systems where text or images have been converted into vector "embeddings" by a machine learning model.

The naive way to answer a nearest-neighbor query is to compute the distance from the query to every single stored vector and keep the k smallest. This is correct but slow: with a million stored vectors, every query touches every one of them. A vector database earns its name by going faster than this through clever indexing. The technique you will implement is called the *inverted file index*, or IVF: cluster all the stored vectors into groups using k-means, remember each group's center, and at query time only search the few groups whose centers are closest to the query. The result is a query that touches only a small fraction of the stored vectors and is therefore much faster, at the cost of occasionally missing a true nearest neighbor that lives in a group the search did not visit. Your benchmark will measure exactly this tradeoff between speed and accuracy.

You are not building a production vector database. There is no SQL, no schema, no metadata, no filtering, no concurrent updates beyond what is needed to serve multiple clients, no persistence beyond a simple snapshot file, and only one distance function. Your effort goes into four things: representing vectors compactly in memory, computing brute-force nearest neighbors as a correctness baseline, implementing k-means and using its output to build the IVF index, and demonstrating the speed/accuracy tradeoff in a benchmark.

2 What the Final Product Looks Like

When your project is complete, the server can be started from the command line, and a separate client can add vectors to it and query them back. A session looks like this. The example uses 4-dimensional vectors so the values are short enough to type; the benchmark in Phase 3 uses 64-dimensional vectors loaded from a file.

```
$ make
$ ./vdb --data ./vdata --dim 4 --port 5556
vdb started on port 5556, dimension 4, data directory ./vdata

[in another terminal]
$ ./vdb-cli localhost 5556
connected to vdb at localhost:5556

> ADD 1 0.10 0.20 0.30 0.40
OK
> ADD 2 0.15 0.25 0.35 0.45
OK
> ADD 3 0.90 0.10 0.10 0.20
OK
> ADD 4 0.85 0.15 0.05 0.25
OK
> ADD 5 0.50 0.50 0.50 0.50
```

```
OK

> SEARCH 0.12 0.22 0.32 0.42 3 BRUTE
1  0.04   0.10 0.20 0.30 0.40
2  0.06   0.15 0.25 0.35 0.45
5  0.49   0.50 0.50 0.50 0.50
(3 results, mode=BRUTE, scanned 5)

> BUILD
Building IVF index...
  vectors:    5
  clusters:   2
  iterations: 4
  done in 0.001s.
OK

> SEARCH 0.12 0.22 0.32 0.42 3 IVF 1
1  0.04   0.10 0.20 0.30 0.40
2  0.06   0.15 0.25 0.35 0.45
5  0.49   0.50 0.50 0.50 0.50
(3 results, mode=IVF, nprobe=1, scanned 3)

> STATS
dimension:        4
total vectors:    5
index built:      yes
clusters:         2
cluster sizes:    3, 2

> SAVE
Saved 5 vectors and IVF index to ./vdata/snapshot.vdb (412 bytes).

> QUIT
```

The interesting line is the second `SEARCH`. Both queries return the same three vectors (a hand-typed example with only five points is not enough to expose the speed-vs-accuracy tradeoff, but it demonstrates the API). Look at the trailing parenthetical: the brute-force search reported `scanned 5`, and the IVF search reported `scanned 3`. With only five vectors and an `nprobe` of 1, the index visited just one cluster of three vectors instead of all five. The benchmark in Phase 3 (Section 7) will scale this up to 50,000 vectors and show speedups of ten times or more, along with recall measurements demonstrating that the IVF index almost always returns the correct answer.

3 The Wire Protocol

Communication between client and server is plain text, one command per line, terminated by a newline. The server responds with one or more lines per command. These are the only commands you must support.

3.1 ADD

```
ADD id v_1 v_2 v_3 ... v_D
```

Insert one vector. `id` is a positive 64-bit integer that the user supplies and that the server uses to refer to this vector in query results. `v_1` through `v_D` are D floating-point numbers, where D is the dimension the server was started with. A wrong number of components is a protocol violation. Adding a vector with an `id` that already exists overwrites the previous one. The server responds with `OK` or an error.

3.2 SEARCH

```
SEARCH v_1 v_2 v_3 ... v_D k mode [nprobe]
```

Return the k stored vectors closest to the query vector. `mode` is either `BRUTE` or `IVF`. In `BRUTE` mode the

server scans every stored vector. In `IVF` mode, an additional integer `nprobe` (the number of clusters to visit) follows; the IVF index must already have been built (otherwise the server returns an error). The response is one line per result, in order of increasing distance, formatted as:

```
id   distance   v_1 v_2 ... v_D
```

followed by a final summary line:

```
(N results, mode=MODE, [nprobe=P, ]scanned S)
```

where `S` is the number of vectors actually scanned during the query. This number is the most important diagnostic the system produces, because it directly explains the cost of the query.

3.3 BUILD

```
BUILD
```

Run k-means on the currently stored vectors and produce the IVF index. The number of clusters is chosen automatically as $\lfloor\sqrt{N}\rfloor$, where N is the current number of stored vectors; this is the standard rule of thumb for IVF. The response includes the cluster count, the number of k-means iterations performed, and the total time taken.

After `BUILD`, subsequent `ADD` commands are stored as before, but they are inserted into the existing IVF index by assigning them to the nearest existing cluster (this is described in Section 6.4). They will be visible to both `BRUTE` and `IVF` searches. Re-running `BUILD` reclusters everything from scratch.

3.4 SAVE and LOAD

```
SAVE
LOAD
```

`SAVE` writes a snapshot of all stored vectors and the IVF index (if built) to a single binary file in the data directory. `LOAD` reads it back into the server, replacing the current state. The exact file format is described in Section 6.5. `LOAD` also runs automatically on server startup if a snapshot file exists.

3.5 STATS and QUIT

```
STATS
QUIT
```

`STATS` prints the dimension, total vector count, whether the IVF index has been built, the number of clusters, and the size of each cluster. `QUIT` disconnects the client; the server keeps running.

4 Architectural Overview

The server has three components. Keep them strictly separated.

```
            [ TCP connections ]
                    |
                    v
          +----------------------+
          |  Network layer       |  accept connections, parse
          |  (Section 4)         |  commands, dispatch, format
          +----------------------+
                    |
                    v
          +----------------------+
          |  Vector store        |  flat array of vectors plus
          |  (Section 5)         |  brute-force search
          +----------------------+
```

```
                    |
                    v
          +----------------------+
          |  IVF index           |  k-means clustering and
          |  (Section 6)         |  bucket-based search
          +----------------------+
```

5 The Vector Store and Brute-Force Search

This is the simpler half of the project and the foundation for everything else. The IVF index in Section 6 is built on top of this store, and the brute-force search you implement here will serve as the ground truth for measuring the IVF index's accuracy.

5.1 Storage Layout

The server is started with a fixed dimension D. Every stored vector has exactly D components. Store all vectors in a single flat array of `float` (32-bit) values: vector i occupies positions $i \cdot D$ through $i \cdot D + D - 1$ of the array. In parallel, maintain an array of 64-bit integer ids, where id i corresponds to vector i. Use a hash map from external id (the value the user supplied with `ADD`) to internal index i for fast lookups during overwrite-on-duplicate-id semantics.

This layout has two virtues. First, it is cache-friendly: brute-force search reads vectors sequentially from contiguous memory, which the CPU's hardware prefetcher handles efficiently. Second, it is trivially serializable: the entire vector array can be written to disk as one block of bytes.

In C, allocate the array as `float* vectors = malloc(N * D * sizeof(float))` and grow it with `realloc` as more vectors arrive. In C++, `std::vector<float>` works the same way. Do not store each vector as a separately allocated array; the indirection wastes memory and destroys cache locality.

5.2 Distance Function

The distance between two vectors a and b in this project is squared Euclidean distance:

$$d^2(a, b) = \sum_{i=1}^{D} (a_i - b_i)^2$$

You compute the squared distance, not the actual distance, because the square root is monotonic and unnecessary: if $d^2(a, x) < d^2(a, y)$ then also $d(a, x) < d(a, y)$, so for ranking purposes the square is sufficient. Skipping the square root is faster and slightly more numerically stable. When you print distances in query results, you may print either the squared distance or its square root, but be consistent and document your choice.

The function itself is a tight loop:

```
float distance_sq(const float* a, const float* b, int D) {
    float sum = 0.0f;
    for (int i = 0; i < D; i++) {
        float diff = a[i] - b[i];
        sum += diff * diff;
    }
    return sum;
}
```

This is the function the entire project spends most of its CPU time inside. You may experiment with optimizing it (loop unrolling, compiler intrinsics) as a personal exercise, but the project does not require it. A straightforward implementation is fast enough for the benchmark.

5.3 Brute-Force k-NN

A brute-force k-NN search computes the distance from the query to every stored vector and returns the k smallest. The natural data structure for tracking "the k smallest seen so far" is a max-heap of size k: when a new vector's distance is smaller than the heap's largest element, pop the largest and push the new one. After scanning all vectors, the heap contains the k nearest, in unsorted order; pop them one at a time into a sorted array.

In C++, `std::priority_queue` is a max-heap by default and is exactly what you want. In C, write a small heap of fixed size k. Both versions are short.

6 The IVF Index

This is where the actual vector database lives. The index is built from a complete vector store by clustering all the vectors into groups, and is queried by visiting only the groups whose centers are closest to the query.

6.1 k-Means Clustering

k-means is the standard algorithm for partitioning N vectors into K groups such that the total distance from each vector to its group's center is small. It is iterative and very simple. The version you will implement is called Lloyd's algorithm:

1. Pick K initial centroids by selecting K vectors at random (without replacement) from the dataset.
2. For each of the N vectors, find the closest centroid and record its index. This is the *assignment step*.
3. For each of the K centroids, recompute it as the mean of all vectors that were assigned to it in step 2. This is the *update step*.
4. If any vector's assignment changed during step 2, go back to step 2. Otherwise, the algorithm has converged.
5. As a safety measure, cap the loop at 50 iterations even if convergence has not been reached.

For this project, K is set automatically to $\lfloor\sqrt{N}\rfloor$. With $N = 50{,}000$ vectors, this gives $K = 223$ clusters. With $N = 1{,}000{,}000$, it gives $K = 1{,}000$. The choice of $\sqrt{N}$ is the standard rule of thumb for IVF and gives a good balance between cluster size and cluster count: the work to find the nearest centroid is $O(K) = O(\sqrt{N})$, and the work to scan one cluster is on average $O(N/K) = O(\sqrt{N})$, so the total query work is $O(\sqrt{N})$ in the best case — a square-root speedup over brute force.

A subtle point: the assignment step is itself a brute-force k-NN search where $k = 1$. Reuse your distance function, but do not reuse the heap-based search; for a single nearest neighbor a simple "best so far" variable is enough.

Empty clusters are possible: a centroid that ended up far away from every vector may have nothing assigned to it. Handle this by leaving such a centroid in place (the cluster simply remains empty in the resulting index). Do not crash, do not delete the cluster, do not pick a new random centroid.

6.2 Index Layout

After clustering, the IVF index is just two pieces of data:

- A flat array of K centroids, each D floats: the cluster centers.
- For each cluster, a list of internal indices of the vectors assigned to it. The simplest representation is a `std::vector<std::vector<int>>` (or in C, an array of K dynamically allocated arrays).

That is the entire index. Notice what is not in the index: the vectors themselves are still stored only in the flat array from Section 5.1. The IVF index references them by integer index. This is intentional and saves a lot of memory.

6.3 IVF Search

A search with the IVF index, given query vector q and parameters k and `nprobe`, proceeds as follows:

1. Compute the squared distance from q to every centroid. There are K centroids, so this is K distance computations.
2. Pick the `nprobe` centroids with the smallest distances. (Use a max-heap of size `nprobe`, exactly the same data structure you used for brute-force k-NN.)
3. For each picked centroid, iterate over the list of vector indices in that cluster. For each one, look up the actual vector in the flat store and compute its distance to q. Maintain a max-heap of size k for the running top-k results.
4. Return the contents of the top-k heap, sorted by distance.

The number of vectors actually scanned in step 3 is at most `nprobe` $\cdot\, \overline{N/K}$ where $\overline{N/K}$ is the average

cluster size. Compared to brute force, which scans all N vectors, this is a factor of approximately $K/$`nprobe` speedup. With $K = 223$ and `nprobe` $= 10$, that is a factor of about 22. Increase `nprobe` for better recall at the cost of speed; decrease it for the opposite.

6.4 Incremental Insertion

After `BUILD`, new vectors arriving via `ADD` need to be inserted into the existing index. The simplest correct approach is: compute the distance from the new vector to every existing centroid, find the nearest one, and add the vector's internal index to that cluster's list. Do not move existing vectors; do not recompute centroids. Over time this degrades the index quality slightly because clusters drift away from being optimal, but for the data sizes in this project the degradation is negligible. Re-running `BUILD` clusters everything from scratch and produces a fresh, optimal index.

6.5 The Snapshot File Format

`SAVE` produces a single binary file. All integers are little-endian. Floats are stored as IEEE 754 binary32.

```
Offset  Size               Field            Description
------  ----------------   ---------------  ----------------------------
  0     4                  magic            ASCII bytes 'V','D','B','1'
  4     4                  version          uint32, must be 1
  8     4                  dimension (D)    uint32
 12     4                  vector_count(N)  uint32
 16     4                  cluster_count(K) uint32, 0 if no index
 20     N*8                ids              int64[N]
 ...    N*D*4              vectors          float[N*D]
 ...    K*D*4              centroids        float[K*D] (omitted if K=0)
 ...    K*4 + total*4      cluster_lists    for each cluster: uint32 size,
                                            then size uint32 indices
```

The file is written via the temporary-file-and-rename pattern (write to `snapshot.vdb.tmp`, then `rename()` it to `snapshot.vdb`) so that a crash mid-write cannot leave a half-written file in place. `LOAD` reads the file in the same order, allocating arrays of the sizes recorded in the header.

7 Phase Structure

7.1 Phase 1 — Network Layer, Vector Store, and Brute-Force Search

The goal of Phase 1 is a running TCP server that supports `ADD`, `SEARCH` in `BRUTE` mode, `STATS`, and `QUIT`, with all data held in memory and no persistence.

Build these components:

- A TCP server that accepts multiple concurrent clients. One thread per client (`pthread_create` or `std::thread`) is the simplest approach; an `epoll`-based server is acceptable but not required.
- A line-based protocol parser that reads one command per line and dispatches to the appropriate handler.
- The flat vector store: an array of floats, an array of ids, and a hash map from external id to internal index. Protect the store with a single mutex held during `ADD` and during the read phase of every search.
- The brute-force `SEARCH` handler: scan the store, maintain a max-heap of size k, sort the heap into the result.
- The `STATS` and `QUIT` handlers.
- A simple command-line client (`vdb-cli`) that opens a TCP connection, reads lines from stdin, sends them to the server, and prints replies.

At the end of Phase 1, the example session in Section 2 should work for everything except `BUILD`, `SEARCH IVF`, `SAVE`, and `LOAD`. Multiple clients connecting simultaneously must not interfere with each other.

7.2 Phase 2 — k-Means and the IVF Index

The goal of Phase 2 is to build the IVF index and use it to answer queries.

Build these components:

- A k-means implementation: random initial centroid selection, the assignment-update loop, the convergence check, and the 50-iteration cap.
- The IVF index data structure: the centroid array and the per-cluster index lists.
- The `BUILD` command handler that runs k-means and produces the index, with timing and iteration count reported in the response.
- The `SEARCH IVF` command handler that uses the index. The output must include the `scanned` count so the difference between brute force and IVF is visible.
- Incremental insertion logic for `ADD` after `BUILD`: find the nearest centroid and append to its cluster list.
- Updates to `STATS` to report cluster count and per-cluster sizes.

At the end of Phase 2, the full example session in Section 2 should work except for `SAVE` and `LOAD`. To verify correctness, take a small dataset (a few hundred vectors), run a brute-force search and an IVF search with a high `nprobe` (large enough to visit every cluster), and confirm they return identical results. Then progressively reduce `nprobe` and observe the results diverging.

7.3 Phase 3 — Persistence and Benchmark

The goal of Phase 3 is to add the snapshot file format and to produce the benchmark report.

Build these components:

- The snapshot writer: serialize all vectors and the index according to Section 6.5, using the temp-and-rename pattern.
- The snapshot reader: deserialize back into the in-memory structures.
- The `SAVE` and `LOAD` command handlers, plus automatic `LOAD` on startup if a snapshot file exists in the data directory.
- A benchmark program (a separate binary or a script that drives `vdb-cli`) that:
  - Generates 50,000 random 64-dimensional vectors. Each component should be drawn from a normal distribution; document the random seed.
  - Inserts them all into a freshly started server via `ADD`. Measure and report wall-clock time and inserts per second.
  - Runs `BUILD`. Report the time taken and the resulting cluster count.
  - Generates 100 random query vectors from the same distribution.
  - For each query, runs a `BRUTE` search and an `IVF` search with `nprobe` values of 1, 5, 10, and 25. The brute-force results are the ground truth.
  - Computes the average wall-clock time per query for each mode and `nprobe` value.
  - Computes the recall@10 for each `nprobe` value: for each query, the fraction of the 10 brute-force nearest neighbors that also appear in the IVF top 10. Average over the 100 queries.
  - Prints a results table with columns: mode, `nprobe`, average query time (ms), average recall@10, and average vectors scanned per query.

A successful submission demonstrates the speed/accuracy tradeoff clearly. The IVF index with `nprobe` = 1 should be at least 10x faster than brute force, with recall above 0.5. With `nprobe` = 10, recall should exceed 0.9, while still being several times faster than brute force. The exact numbers depend on the data distribution and the choices you made in clustering, and your design document must include the full results table along with a paragraph explaining the tradeoff your numbers show.

8 Deliverables and Submission

8.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation, including both the server (`vdb`) and the client (`vdb-cli`).
- A `Makefile` (or `CMakeLists.txt`) that builds both binaries with a single command on a recent Ubuntu LTS using only the standard C/C++ toolchain.
- A `README.md` explaining how to build and run the project, the list of supported commands, and any known limitations.

- A `design.pdf` document of at most 8 pages describing the three components, the data layout for the vector store and the IVF index, the k-means implementation including the cluster count rule, the snapshot file format (essentially repeating Section 6.5 in your own words), the concurrency approach, and a discussion of the benchmark results. The benchmark discussion is the most important section: include the full results table and explain what the recall and timing numbers show about the speed/accuracy tradeoff.
- A `benchmark/` directory with the benchmark program, a script that runs it end to end against a freshly started server, and the results of your final run.
- A `tests/` directory containing simple test cases that exercise the protocol parser, the distance function, the k-means implementation (verify it converges on a small handcrafted dataset), and the snapshot reader and writer (round-trip test: save, load, verify everything matches).

8.2 Archive Naming

`Group[Number]_Project02_VectorDB.zip`, for example `Group17_Project02_VectorDB.zip`.

8.3 Demonstration

Each group will present a live demonstration during the week following the submission deadline. The demonstration consists of three parts. First, a brief walkthrough of the three components using the design document. Second, a live execution of the benchmark against a freshly built server, with the instructor/TAs free to modify the benchmark parameters (number of vectors, dimension, `nprobe` values) on the spot. Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component of the implementation. All group members must be present. Missing the demonstration results in a zero for Phase 3.

9 Bonus Opportunities

Each of the following is independent and may be attempted in any combination. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points total. To claim bonus credit, your design document must include a dedicated section explaining which bonus items you attempted and how you verified them.

9.1 Cosine Similarity (up to 5%)

Add a server-side flag (or `-metric` command-line option) that switches the distance function to cosine distance: $1 - \frac{a \cdot b}{\|a\|\|b\|}$. The natural way to implement this efficiently is to normalize every vector to unit length on insertion, so that cosine distance reduces to $1 - a \cdot b$, which is computed by a dot-product loop the same length as your existing Euclidean loop. Your benchmark must include a run with cosine distance and a brief discussion of what changes about the recall numbers and why.

9.2 k-means++ Initialization (up to 5%)

Replace the random initial centroid selection with the k-means++ algorithm. The first centroid is chosen uniformly at random; each subsequent centroid is chosen with probability proportional to the squared distance from the nearest already-chosen centroid. This produces dramatically better initial centroids, especially for clustered datasets, and reduces the number of iterations needed for convergence. Report the comparison (iterations and final clustering quality) in your design document.

9.3 Multi-Probe Refinement (up to 5%)

The IVF query algorithm in Section 6.3 picks the `nprobe` closest centroids. A more sophisticated approach picks the closest `nprobe`, scans them, and then conditionally scans additional clusters if the current top-k has not yet stabilized (for example, if the worst result in the top-k heap is still farther from the query than the nearest unvisited centroid). Implement this and demonstrate in your benchmark that it improves recall at the cost of a small increase in scanned-vector count.

10 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing vector search library or its components. This includes but is not limited to FAISS, hnswlib, Annoy, ScaNN, NMSLIB, USearch, Qdrant, Milvus, and their language bindings. Even using such a library only for "the k-means part" or "just the index" is not permitted.
- Use of any existing machine learning library that includes a k-means implementation. This includes scikit-learn, mlpack, dlib's clustering routines, and their equivalents. You must implement k-means yourself.
- Use of any existing binary serialization library (Protobuf, Cap'n Proto, FlatBuffers, MessagePack). The snapshot file format is specified in Section 6.5 and must be implemented with direct reads and writes.
- Use of text-based storage (JSON, CSV, XML) as the persistent format. The snapshot file must be the binary format described.
- Use of Python at any layer of the system, including test harnesses and benchmark drivers. Tests and scripts must be written in C, C++, shell, or JavaScript.
- Storing each vector as its own dynamically allocated array (one `malloc` per vector). The flat-array layout described in Section 5.1 is required.
- Submitting code whose commit history shows large, infrequent commits. Your git history should reflect the incremental development of a system of this size; histories with three commits totaling thousands of lines will be scrutinized for academic integrity.

11 Required Reading

The first item is short and required reading before you start. The remaining items are reference material to consult during implementation.

1. Pinecone, "Faiss: The Missing Manual" — specifically the chapter on "Inverted File Index." A practical, well-illustrated explanation of exactly the index you will build. The article is available freely on Pinecone's documentation site. Read it before Phase 2.
2. Lloyd, S. P. "Least Squares Quantization in PCM." *IEEE Transactions on Information Theory*, 1982. The original paper describing the k-means algorithm. The notation is dated but the algorithm is exactly what you will implement; read Section II.
3. Arthur, D. and Vassilvitskii, S. "k-means++: The Advantages of Careful Seeding." *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms*, 2007. Optional. Required reading if you attempt the k-means++ bonus. Section 2 is enough.
4. Jégou, H., Douze, M., and Schmid, C. "Product Quantization for Nearest Neighbor Search." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011. Optional. Background on the more sophisticated indexing techniques (PQ, IVFADC) used by production vector databases. The IVF you are building in this project is the foundation that PQ is built on top of.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday, 11:30 AM – 2:30 PM